AI Engineering and Claude Architect · Part IV, Production

AI Production Engineering, Lesson 3

Evals: Datasets, Hallucination and CI Gates

The pipeline scored 98.3% overall, and the team prepared to remove human review. One document type was getting a key field right less than half the time. This lesson holds the exam's content on calibration and review.

Module	AI10 of 13
Lesson	3 of 7
Prerequisites	AI01 L2, AI02 L6, AI05 L6, AI08 L7
Exam relevance	**Core for TS 5.5.** Also TS 4.1, 4.4, 4.6, 5.6
Reading time	~45 minutes

Contents

1. Evals

Lesson 3 of 7 · AI Production Engineering

Evals

Key idea

An eval turns "seems better" into a number with an error bar, and an average hides the segment that's about to hurt you. Build from real failures, grade outcomes, measure by segment, calibrate confidence on labelled data, and keep sampling after you automate. A suite too small to detect the drop you care about is theatre.

> *Before: precision and recall, hallucination, retrieval metrics, testing AI code. Teaches: eval vocabulary, graders, segment analysis, calibration, stratified sampling, gate statistics, judge validation, and hallucination measures. Used by: L2, L6, AI13.*

The hook: 98.3% overall

A document extraction pipeline handles invoices, receipts, and a few handwritten forms. It scores well, and the team proposes removing human review. **The data below is synthetic, generated from a seed with accuracy I chose,** so it demonstrates the method and measures no model. Real output from `EvalLab.java`:

```
Overall accuracy on 20,000 test fields: 98.3%
By segment:
  handwritten/date       n=  198  accuracy  81.3%  95% CI  75.3- 86.1%
  handwritten/tax_id     n=  202  accuracy  46.5%  95% CI  39.8- 53.4%
  handwritten/total      n=  197  accuracy  79.7%  95% CI  73.5- 84.7%
  invoice/date           n= 5592  accuracy  99.5%  95% CI  99.3- 99.6%
  invoice/tax_id         n= 5774  accuracy  99.4%  95% CI  99.2- 99.6%
  invoice/total          n= 5595  accuracy  99.5%  95% CI  99.3- 99.7%
  receipt/date           n=  785  accuracy  98.0%  95% CI  96.7- 98.7%
  receipt/tax_id         n=  814  accuracy  97.9%  95% CI  96.7- 98.7%
  receipt/total          n=  843  accuracy  97.0%  95% CI  95.7- 98.0%
```

Handwritten forms are 3% of volume, so they barely move the average. And the tax ID on those forms is right 46.5% of the time.

97% overall can still mean a segment is failing. That's the exam guide's own example in TS 5.5, and it's why you measure by document type and field before reducing review.

Intuition: the average temperature in a hospital

A hospital's average patient temperature is 37 degrees. That's reassuring, and it would still be 37 if one ward had hypothermia and another had a raging fever. Nobody runs a hospital on that number, and they read each chart.

An eval's overall score is that average. The charts are your segments, and each segment gets its own reading.

Mechanism

Vocabulary

Anthropic's engineering article Demystifying evals for AI agents (January 2026) gives a clean set of terms. I'll use them throughout.

Term	Meaning
Task	One test with defined inputs and success criteria
Trial	One attempt at a task. Several trials handle variability
Grader	Logic that scores some aspect of performance
Transcript	The full record of a trial, including tool calls and reasoning
Outcome	The final state of the environment, such as a reservation in the database
Harness	What runs evals end to end
Suite	A collection of tasks measuring a capability or behaviour

Capability evals ask what the agent can do, and start at a low pass rate. **Regression evals** ask whether it still handles past tasks, and should approach 100%. When a capability suite saturates, it can graduate into regression.

Building a dataset

The article's advice is direct: about 20–50 tasks drawn from real failures is enough to begin. Start with manual pre-release checks, your bug tracker, and support queues.

Rule	Why
Start from real failures	They're real, and they matter
Mirror the real distribution	Anthropic's guide: "Design evals that mirror your real-world task distribution."
Include edge cases	Irrelevant, too-long, and ambiguous inputs
Balance both directions	Test when a behaviour should occur and when it shouldn't, or you'll optimize one side
Write unambiguous tasks	A 0% pass rate across many trials often means a broken task
Keep held-out data	You can overfit a prompt to an eval (AI01 L2)
Keep segment labels	Without them, you can't do any of what follows
Isolate each trial	Shared state causes correlated failures

Volume or quality? Anthropic's guide to evaluations says to prefer volume: "More questions with slightly lower signal automated grading is better than fewer questions with high-quality human hand-graded evals." The two statements fit together. Begin with 20–50 real failures, since a start beats none, and grow, because of the statistics below.

Graders

The article compares three kinds:

Grader	Strengths	Weaknesses	Use for
Code	Fast, cheap, objective, reproducible	Brittle, misses nuance	State checks, schemas, exact fields, tests
Model	Flexible, handles open-ended output	Non-deterministic, needs human calibration	Tone, faithfulness, rubrics
Human	The best quality	Slow and costly	Calibrating the others, and hard judgment

Anthropic's guide lists specific methods: exact match, cosine similarity for consistency, ROUGE-L for summaries, an LLM Likert scale for tone, and LLM binary classification. It also advises: "Generally best practice to evaluate with a different model from the one used to generate the evaluated output."

Grade outcomes, not paths. The article's phrase is "grade what the agent produced, not the path it took." A refund test should check that one correct refund exists in the database, rather than asserting the exact tool order. Agents find valid routes you didn't think of, and step-sequence tests fail on them. AI08 L7 made the same point about asserting effects rather than words.

Read transcripts. A number can't tell you whether a failure was fair. Reading them is how you find a broken task, or a grader that's being gamed.

Validating a judge

A model grader is a classifier, and it needs testing against people. Here's a trap: raw agreement is misleading on imbalanced data. Real output, with counts I invented so both reach 90%:

```
Judge against humans, 200 answers: raw agreement 90.0%, kappa 0.45
Judge against humans, balanced:    raw agreement 90.0%, kappa 0.80
```

Cohen's kappa measures agreement beyond chance. Both judges agree with people 90% of the time. In the first case, most answers are good, so a judge saying "good" to nearly everything also reaches a high raw number. The arithmetic lives in `EvalLab.kappa`, and the point is to look further than the raw percentage. Keep a human-labelled set, re-check when you change the judge's prompt or model, and watch for judges being generous or harsh on a particular segment.

Calibration: what "90% confident" is worth

TS 5.5 teaches field-level confidence scores calibrated on labelled validation sets. Here's why calibration is needed:

```
Calibration on the validation set:
  stated   0- 70%  n=   14  avg stated  67.8%  real   0.0%
  stated  70- 80%  n=  439  avg stated  77.6%  real  69.9%
  stated  80- 85%  n= 1695  avg stated  83.0%  real  93.0%
  stated  85- 90%  n= 4306  avg stated  87.8%  real  97.9%
  stated  90- 95%  n= 6240  avg stated  92.5%  real  99.7%
  stated  95-100%  n= 7306  avg stated  98.1%  real  99.9%
  expected calibration error: 6.1 points
```

Stated confidence **isn't** a probability, though it does carry information: higher stated means higher real. Here, it understates the truth at the top and overstates it at the bottom. In real systems, the error can go either way, so only labelled data can tell you. AI02 L6 explained why a model's self-report isn't reliable by itself.

Now use it to set a review threshold, with the choice made on validation data, and the result measured on separate test data:

```
Routing by confidence, threshold chosen on validation only:
  naive: trust 90% as stated     threshold 0.900: auto 68.4%, accepted accuracy 99.78%, handwritten tax_id 85.7%
(n=77)
  calibrated for 99.5% accepted  threshold 0.865: auto 84.9%, accepted accuracy 99.64%, handwritten tax_id 79.0%
(n=105)
  same, handwritten sent to people threshold 0.865: auto 85.5%, accepted accuracy 99.83%, handwritten tax_id none
```

Three lessons, from three rows:

1. **Calibration can save work.** Taking "90%" at face value sent about a third of fields to people. The calibrated threshold reaches the target accuracy, with 85% automated.
2. **A global threshold still leaks the weak segment.** Row two looks great overall at 99.64%, while the handwritten tax ID, in the accepted set, is 79%. **Calibrate per segment, or route weak segments to people whatever the score.**
3. **Row three does that,** and overall quality goes up as well. Its "auto" share is measured on the non-handwritten fields.

The review workflow

Combining this with TS 5.5:

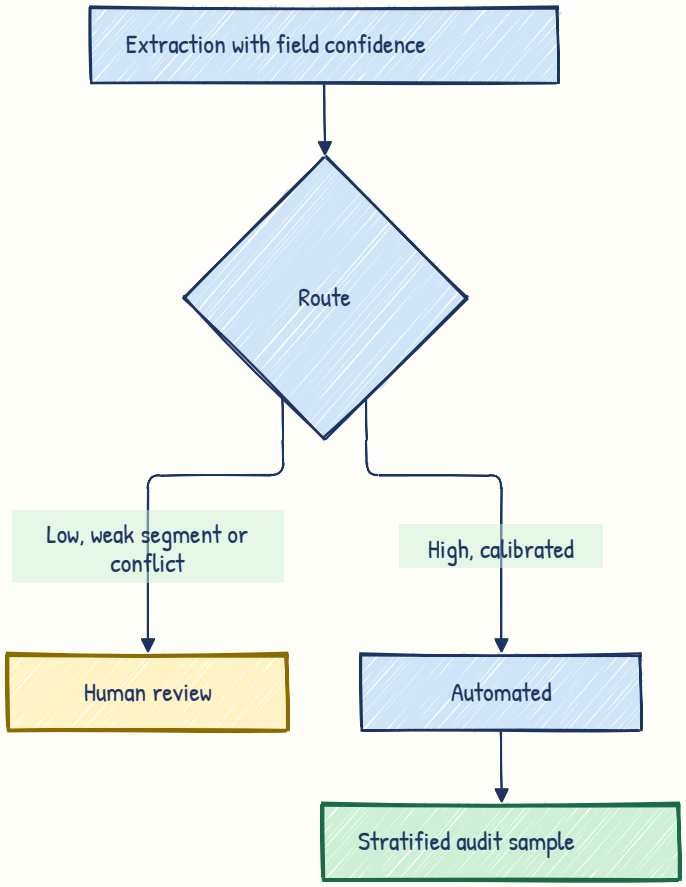


Figure 1. Confidence and segment decide the route. A sample of what's automated keeps measuring, so that drift shows up.

From the guide: route **low confidence** and **ambiguous or contradictory source documents** to people, prioritizing limited reviewer capacity. Corrections then become labelled data, which feeds calibration.

Stratified sampling

After automating, you still need to measure error rates, and to catch new kinds of error. Sampling at random spends most of its budget on the biggest group. Real output, auditing 300 high-confidence fields:

```
Sampling 300 high-confidence fields for human audit:
  invoice      true  99.9% | random n=257 est 100.0% | stratified n=100 est 100.0%
  receipt      true  99.4% | random n= 34 est 100.0% | stratified n=100 est 100.0%
  handwritten  true  91.6% | random n=  9 est 66.7% | stratified n=100 est  87.0%
```

A random sample took **nine** handwritten fields, and estimated 66.7%, from nine points. The stratified sample has a hundred per stratum, for the same budget. **Stratify so that small, risky groups get enough samples to measure.**

Two cautions. To report an overall figure from a stratified sample, weight each stratum by its real share. And even 100 samples leave a wide interval, as the next section shows.

How big must a suite be?

Pass rates carry noise, and small suites carry a lot. For a measured 90%:

```
95% interval for a measured 90% pass rate:
     20 tasks:  69.9- 97.2% (width 27.3 points)
     50 tasks:  78.6- 95.7% (width 17.0 points)
    100 tasks:  82.6- 94.5% (width 11.9 points)
    200 tasks:  85.1- 93.4% (width  8.4 points)
    500 tasks:  87.1- 92.3% (width  5.3 points)
   1000 tasks:  88.0- 91.7% (width  3.7 points)
```

(These are Wilson intervals.) A 20-task suite can't tell 75% from 95%. That doesn't contradict "start with 20–50." It's excellent for finding broken things. It can't detect a two-point regression.

CI gates, and their error rates

A "must pass 90%" gate is a statistical test, whether you meant it to be or not. I ran it 10,000 times per case:

```
A gate of 90% on 50 tasks, run 10,000 times:
  true rate 92%: gate passes  78.5% of runs
  true rate 90%: gate passes  61.3% of runs
  true rate 88%: gate passes  42.7% of runs
  true rate 85%: gate passes  22.6% of runs
The same gate on 500 tasks:
  true rate 92%: gate passes  95.4% of runs
  true rate 90%: gate passes  53.4% of runs
  true rate 88%: gate passes   9.9% of runs
  true rate 85%: gate passes   0.1% of runs
```

With 50 tasks, a real regression to 85% still passes more than one run in five, and a healthy 92% fails about one in five. Engineers learn to rerun until green, and the gate loses its meaning. This is what makes people distrust evals.

Practical gate design, as an engineering recommendation:

Practice	Why
Hard gates on must-never cases	Safety and policy cases should always pass (AI08 L7's segment gate)
Compare with the baseline on the same tasks	Pairing removes much of the noise between tasks
Run several trials on flaky tasks	Separates variability from failure
Track trends	A slow slide is invisible to a single gate
Tier the suites	Small and fast per commit, large and slow per release
Never rerun until green	It defeats the gate. Investigate instead

Cost. Each run spends money. The Batch API at half price suits large eval suites, since nobody's waiting.

Measuring hallucination

AI02 L6 explained why it happens, AI03 L3 how prompts reduce it, and AI05 L6 how to measure faithfulness in RAG. Here's the measurement summary:

Situation	Measure	How
Fact known to you	Correctness	Code check against ground truth
Answer should come from documents	Faithfulness	Does each claim have support? Citations help (AI05)
Answer should come from tools	Fact check	Compare figures with tool results (AI09 L5)
Question has no answer	Abstention	Include unanswerable tasks, and expect "I don't know"
Citations	Validity	Does the document exist, and does the quote appear?
Open-ended	Claim extraction, then checks	Model-based, and validated against people

Unanswerable questions are the cheapest high-value test, in my judgment. Many suites contain only answerable ones, so a model that invents answers gets full marks.

Evals aren't the only layer

The article uses a Swiss Cheese analogy: no single layer catches everything.

Method	Strength	Weakness
Automated evals	Fast, reproducible, on every change	False confidence if they don't match real use
Production monitoring	Ground truth and drift	Reactive
A/B tests	Real user outcomes	Slow, and needs traffic
User feedback	Finds what you didn't think of	Sparse, and skews severe
Transcript review	Builds intuition	Doesn't scale
Human studies	Calibrate model graders	Costly

The loop is what matters most: failures in production become tasks in the suite.

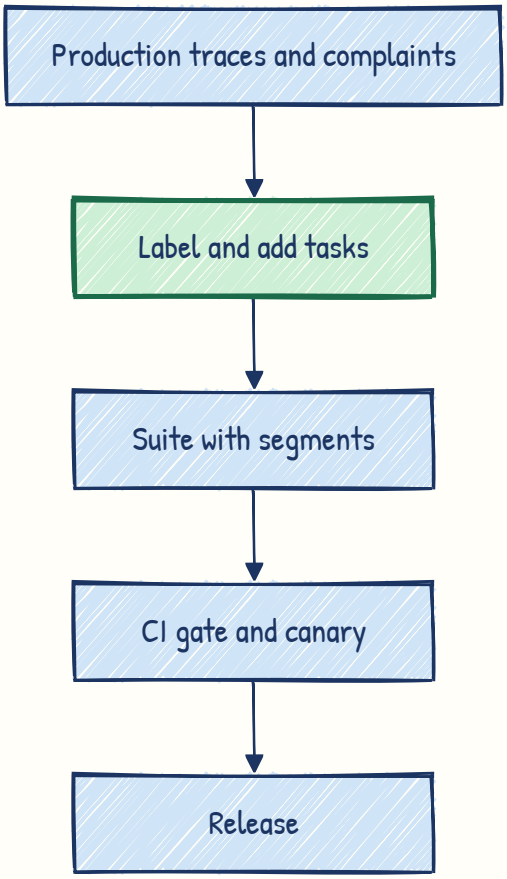


Figure 2. Production feeds the suite. The suite gates the next release. It only compounds if someone keeps the loop running.

Why the obvious fixes fall short

Obvious idea	Why it falls short
Report overall accuracy	Hid a 46.5% field
Trust the model's confidence	Needs labelled calibration
Sample at random	Nine samples from the risky group
A hard 90% gate on 50 tasks	Passes 85% more than one time in five
Only hand-graded evals	Too few to detect drops, and slow to grow
Step-sequence assertions	Fail on valid alternative routes
Same model as judge and subject	Anthropic advises a different model
Rerun until green	The gate stops meaning anything

How Anthropic solved it

Topic	What's published	Source
Agent evals	Vocabulary, graders, capability and regression, roadmap	Demystifying evals
Test design	Task-specific, automate, volume over quality	Evaluation guide
Grading methods	Exact match, cosine similarity, ROUGE-L, LLM graders	Evaluation guide
Multi-grader judgment	Different model for grading	Evaluation guide
Human review	TS 5.5	Exam guide

That's Anthropic's published method. Its internal numbers aren't given in these documents, so this lesson's figures are from my synthetic data.

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: a person tries a few prompts before release.

Improved: a suite of real failures with a CI gate. New problem: the suite drifts from reality, and small suites flip at random.

Production: a dataset with owners and segments, baseline comparison, and sampling from production.

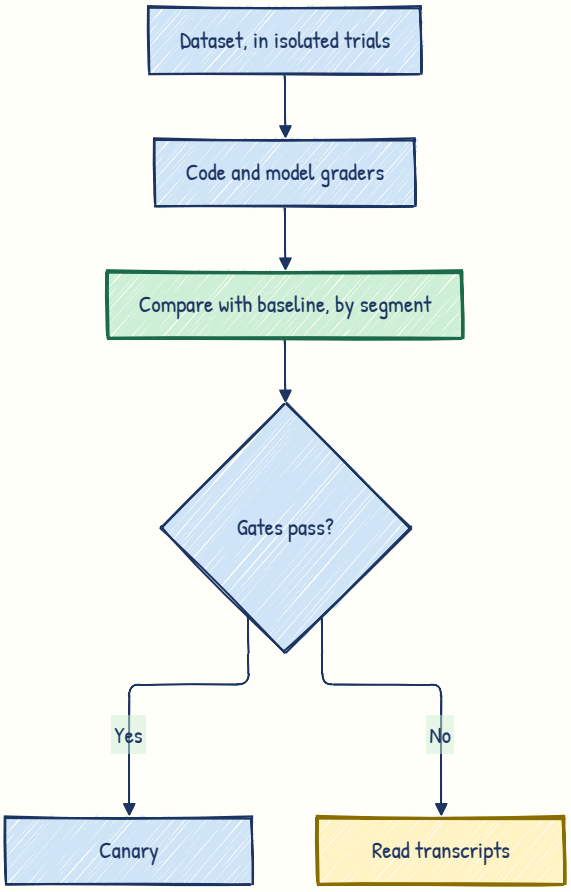


Figure 3. A release runs against the same tasks as its baseline, so that the comparison isn't mostly noise.

Component	Why it exists	How it communicates
Dataset store	Versioned tasks, with segment labels and sources	Git or a database
Harness	Isolated trials, repeated runs	CI, Batch for large suites
Graders	Code first, then validated model graders	Library
Baseline store	Comparison on the same tasks	Per release
Gates	Must-pass cases, and paired comparison	CI status
Production sampler	Stratified audits, new tasks, drift	Queue to reviewers
Calibration job	Keeps thresholds honest	Scheduled, on labelled data
Owner	Suites decay without one	A named team, as the article advises

Failures:

Failure	How you'd notice	What to do
Suite stuck at 100%	No signal	Add harder tasks, as the article advises
Overfit prompt	Eval rises, production doesn't	Held-out data, and fresh production samples
Judge drift	Disagreement with periodic human labels	Re-validate on every change
New document type appears	Audit errors with a new pattern, or drift	Add a stratum, and route it to people
Thresholds go stale after a model change	Audit error rate rises	Recalibrate on each model change
Flaky gate	People rerun, or bypass it	Larger or paired suite
Personal data in the dataset	Audit	AI09 L4: synthetic data or access rules

Java

The per-segment number comes first, and its interval is printed beside it, so that small groups look uncertain:

```
double centre = (p + z * z / (2 * n)) / denominator;
double margin = z * Math.sqrt(p * (1 - p) / n
        + z * z / (4.0 * n * n)) / denominator;
```

This is the Wilson interval, which behaves sensibly at small samples and near 0 or 1, where the textbook formula doesn't.

The threshold search uses **validation data only**, and requires a minimum accepted volume:

```
if (accepted.size() >= 200 && accuracy(accepted) >= target) {
    return threshold;
```

Using test data here would be overfitting the threshold, and a very small accepted set could hit the target by luck.

The quota is the entire difference between random and stratified:

```
if (quota.getOrDefault(key, 0) < budget / 3) {
```

Trade-offs

Decision	Benefit	Cost
Large suite	Detects small drops	Cost per run
Code graders	Reproducible	Brittle, and miss nuance
Model graders	Handle open-ended output	Need validation, and cost money
Lower review threshold	More automation	More errors reach users
Per-segment thresholds	Protects weak groups	Needs labels per segment
Stratified audit	Measures small groups	Needs re-weighting for totals
Strict gate	Fewer regressions ship	More false alarms

When not to invest heavily: a throwaway prototype needs ten checks, and a person reading. Scale up as soon as real users depend on it.

Common mistakes

Removing review because of a good average. Check each document type and field first. This is the TS 5.5 anti-pattern.

Common mistakes

Using a validation set for both choosing and reporting. The number you report will be too good. Keep test data separate.

Common mistakes

Building evals only from answerable questions. You can't detect invented answers without tasks that have no answer.

What to say in an interview

"I'd start from 20 to 50 real failures, with code graders first, and grade outcomes. I report by segment with intervals, because an average can hide a failing field. Model confidence gets calibrated on labelled data, thresholds are chosen on validation data, and weak segments go to people. After automating, I keep a stratified audit sample. And I treat a CI gate as a statistical test: I'd compare with the baseline on the same tasks, and size the suite for the drop I need to detect."

Certification lens

How this lesson maps to CCAR-F

This lesson is exam content for TS 5.5, which covers aggregate metrics masking weak segments, stratified random sampling of high-confidence extractions, field-level confidence calibrated on labelled validation sets, and routing low-confidence or ambiguous documents to review. **Nearby:** explicit criteria and false positives (TS 4.1), validation and retry (TS 4.4), independent review (TS 4.6), and provenance (TS 5.6). Wilson intervals and kappa aren't in the guide. CI gates and tracing tools aren't either.

These are original practice questions, not real or leaked.

Q Q1 · Exam style, scenario: Structured Data Extraction · Medium. Extraction reaches 97% accuracy overall. Management wants to remove human review for fields with high model confidence. What should happen first?

A. Raise the confidence threshold to 99%

B. Switch to a larger model

C. Add few-shot examples, then re-measure the overall figure

D. Measure accuracy by document type and field on a labelled set, before reducing review

A D (TS 5.5). The aggregate can mask poor segments. A adjusts a threshold with no calibration or segment data, B and C may help but don't answer whether automation is safe.

Q Q2 · Exam style, scenario: Structured Data Extraction · Medium. You've automated high-confidence extractions. How do you keep measuring error rates and catch new error patterns?

A. Take a stratified random sample of the high-confidence extractions for human review

B. Review only low-confidence extractions

C. Ask the model to report any uncertainty in its output

D. Wait for customer complaints

A A (TS 5.5). Automated items would otherwise never be checked. B never sees errors in the automated set. C relies on the same uncalibrated signal, and D measures only what customers notice and report.

Q Q3 · Exam style, scenario: Structured Data Extraction · Hard. Reviewer capacity is limited, and contracts sometimes contain conflicting dates. Which routing is best?

A. Send a uniform random 20% to reviewers

B. Accept the most recent date, and don't review

C. Route low-confidence fields and documents with conflicting sources to reviewers, using thresholds calibrated on labelled data

D. Have the model average its confidence, and route if below 50%

A C (TS 5.5, 5.6). It prioritizes reviewers' time where risk is highest, and doesn't silently resolve conflicts. A ignores risk, B discards a conflict, and D uses an uncalibrated number and an arbitrary cut-off.

Q Q4 · Exam style, scenario: Code Review in CI · Medium. Developers ignore an automated review tool because of false positives in one category. What's a good first step?

A. Remove the tool from CI

B. Temporarily disable that high false-positive category while improving its criteria, and keep measuring

C. Ask developers to approve all findings

D. Increase the number of findings per file

A **B** (TS 4.1). False positives destroy trust in all findings, so disabling the noisy category preserves trust in the rest. A throws away value, while C and D make the trust problem worse.

Q Q5 · A team's gate is "pass rate of at least 90% on 50 tasks." A release fails at 88%. They rerun it and it passes at 92%, so they ship. What's wrong, and what would you change?

A At 50 tasks, the interval around 90% is about 17 points wide, so those two runs may be the same system. Rerunning until green makes the gate meaningless. Compare with the baseline on the same tasks, run more trials or a larger suite, and keep hard gates for must-never cases. Investigate the failures, rather than sampling again.

Revision - Lesson 3

→ Overall accuracy can hide a failing segment. Measure by document type and field before reducing review.

→ Vocabulary: task, trial, grader, transcript, outcome, harness, suite. Capability and regression evals.

→ Start with 20–50 real failures, mirror real distribution, balance both directions, keep held-out data.

→ Graders: code, model, human. Grade outcomes, not paths. Grade with a different model.

→ Judges need validating against people. Raw agreement misleads, so also use kappa.

→ Stated confidence isn't a probability. Calibrate on labelled validation data, then test separately.

→ A global threshold can leak a weak segment, so calibrate per segment or route it to people.

→ Stratified sampling gives small risky groups enough samples. Re-weight for totals.

→ Route low confidence and conflicting sources to people, and prioritize limited reviewers.

→ A gate is a statistical test. 50 tasks has an interval about 17 points wide. Never rerun until green.

→ Measure hallucination: correctness, faithfulness, fact checks, abstention, citation validity.

→ Production failures become tasks. Monitoring, A/B tests and review are other layers.

Your notes